

Named Parameters in C# with Examples

In this article, I am going to discuss **Named Parameters in C#** with Examples. Please read our previous article where we discussed [Ref vs Out in C#](#) with Examples. This concept is introduced in C# 4.0.

Named Parameters in C#

According to MSDN, the named arguments enable us to specify an argument for a parameter by matching the argument with its name rather than with its position in the parameter list. And this Named Parameters can be used with methods, indexers, constructors, and delegates.

When we use named arguments, then the arguments are evaluated in the order in which they appear in the argument list, not the parameter list.

Example to Understand Named Arguments in C#:

The Named Arguments in C# free us from matching the order of arguments to the order of parameters in the parameter lists of called methods. The argument for each parameter can be specified by parameter name. Let us understand this with an example. So, what we are going to do is, first we will see the example without using the Named Parameter, and then we will see the same example using the Named Parameters in C#.

Example without using Named Parameters in C#:

Please have a look at the below example. This is a very simple example. Here we have created a class called Employee with one method called AddEmployeeInfo. This AddEmployeeInfo method takes the employee details and then saves the employee data into the database. For the simplicity of this example, we are not writing the logic to add the employee data into the database. Then from the Main method, we are creating an instance of the Employee class and then invoke the AddEmployeeInfo method by passing the required values. Here, we are not using Named Parameters while calling the AddEmployeeInfo method.

```
using System;
namespace NamedParametersDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Employee employee = new Employee();
            employee.AddEmployeeInfo(1001, "Pranaya", true, "1234567890", "IT'
            Console.ReadKey();
        }
    }
    public class Employee
    {
        public void AddEmployeeInfo(long EmpID, string Name, bool IsPermanent,
        Mobile, string Department)
        {
            //Add the logic to Add the employee in the database
            Console.WriteLine("Employee Info Added");
        }
    }
}
```

```

    }
}
}

```

In the above example, while calling the AddEmployeeInfo method, the argument order of what we are passing is important. We need to pass the values of the arguments in the same order in which the parameters are defined in the AddEmployeeInfo method. If we pass the arguments in random order, then we may store the wrong data in the database or we may get some compiler error if the parameter type is not matched. For a better understanding, please have a look at the following example.

```

using System;
namespace NamedParametersDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Employee employee = new Employee();
            employee.AddEmployeeInfo( true, "Pranaya", 1001, "IT", "1234567890");
            Console.ReadKey();
        }
    }
    public class Employee
    {
        public void AddEmployeeInfo(long EmpID, string Name, bool IsPermanent,
        Mobile, string Department)
        {
            //Add the logic to Add the employee in the database
            Console.WriteLine("Employee Info Added");
        }
    }
}

```

Now, in the above example, we are passing the arguments in the wrong order and hence we try to compile the above code, we will get the following compilation error.

	Code	Description
✖	CS1503	Argument 1: cannot convert from 'bool' to 'long'
✖	CS1503	Argument 3: cannot convert from 'int' to 'bool'

See, here it is complaining that Argument 1 cannot convert from bool to int. This is because in the method we are declaring the first parameter as long type and while calling the method we are passing the first parameter as Boolean type and hence we are getting the error. Similarly, we are getting another compilation error stating Argument 3 cannot convert from int to bool. And this makes sense because the third argument in the method is declared as bool and for this parameter, we are passing an int value.

Note: Using named parameters in C#, we can put any parameter in any sequence as long as the name is there. The right parameter value based on their names will be mapped to the right variable. The parameters name must match the method definition parameter names

Can we Pass Arguments in Random Order in C#?

Yes. We can pass arguments in Random order or in any order. For this, we need to use Named Parameters. In Named Parameters, we need to specify the parameter name while calling the method. For a better understanding, please have a look at the below example. Here, it is the same example that the previous one which gives two compilation errors. But with named parameters, now, we are not getting any compilation error. So, we need to type the parameter name, then a colon, and then we need to specify the values.

```
using System;
namespace NamedParametersDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Employee employee = new Employee();
            employee.AddEmployeeInfo(IsPermanent: true, Name: "Pranaya", EmpID: 1234567890, Department: "IT", Mobile: "1234567890" );
            Console.ReadKey();
        }
    }
    public class Employee
    {
        public void AddEmployeeInfo(long EmpID, string Name, bool IsPermanent, string Department, string Mobile)
        {
            //Add the logic to Add the employee in the database
            Console.WriteLine("Employee Info Added");
        }
    }
}
```

Now, with the above changes in place, you will not get any compilation errors.

Can we pass arguments using Both Named and Simple Arguments in a Method call?

Yes, it is possible in C# to use both Named Arguments and general arguments. In this case, The Named argument specifications must appear after all fixed arguments have been specified. For a better understanding, please have a look at the below example. Here, the first two arguments are fixed arguments i.e. EmpID and Name, and then the rest three arguments are specified by using the Named Argument.

```
using System;
namespace NamedParametersDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Employee employee = new Employee();
```

```

        employee.AddEmployeeInfo(1001, "Pranaya", Department: "IT", IsPerm
true, Mobile: "1234567890");
        Console.ReadKey();
    }
}
public class Employee
{
    public void AddEmployeeInfo(long EmpID, string Name, bool IsPermanent,
Mobile, string Department)
    {
        //Add the logic to Add the employee in the database
        Console.WriteLine("Employee Info Added");
    }
}
}

```

But, if you try to use the Fixed arguments after the Named Arguments, then you will get a compilation error. For a better understanding, please have a look at the below example. Here, the first two arguments are fixed arguments, the next two arguments are Named Arguments, and the last one we are using as a Fixed Argument.

```

using System;
namespace NamedParametersDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Employee employee = new Employee();
            employee.AddEmployeeInfo(1001, "Pranaya", IsPermanent: true, Mobi
"1234567890", "IT");
            Console.ReadKey();
        }
    }
    public class Employee
    {
        public void AddEmployeeInfo(long EmpID, string Name, bool IsPermanent,
Mobile, string Department)
        {
            //Add the logic to Add the employee in the database
            Console.WriteLine("Employee Info Added");
        }
    }
}

```

So, when you compiled the above code, you will get the following compilation error. It is clearly saying that Named argument specifications must appear after all fixed arguments have been specified. So, you cannot use Fixed Argument after the Names Argument in C#.

Code	Description
 CS1738	Named argument specifications must appear after all fixed arguments have been specified. Please use language version 7.2 or greater to allow non-trailing named arguments.

Using Named Parameters with Optional Parameters in C#:

When Named Parameters are used with the [Optional Parameters in C#](#), the usability and the function call become easier and also much enhanced. [Optional Parameters in C#](#) are a way of saying that if you do not want to provide a value to a parameter, then we will take a default value and you can skip providing the value of that parameter during the function call.

For a better understanding, please have a look at the following example. See, the optional parameters must have appeared after all the required parameters in the parameter list. Here, we have specified the default value for the IsPermanent parameter as True which makes this parameter an optional parameter. Similarly, we provided the default value of Department as IT and hence this also becomes an optional parameter. Now, it is up to us whether we want to provide the values for optional parameters or not. If we provided, then it will take the provided values else it will take the default values.

```
using System;
namespace NamedParametersDemo
{
    class Program
    {
        static void Main(string[] args)
        {
            Employee employee = new Employee();
            employee.AddEmployeeInfo(EmpID:1001, Name:"Pranaya", Mobile: "1234
Department:"Sales");
            Console.ReadKey();
        }
    }
    public class Employee
    {
        public void AddEmployeeInfo(long EmpID, string Name, string Mobile, bool
IsPermanent = true, string Department = "IT")
        {
            //Add the logic to Add the employee in the database
            Console.WriteLine("Employee Info Added");
        }
    }
}
```

You can see in the above example; that we have provided all the required parameter values plus we have provided the values for Department optional parameter. But we have not provided the value for the IsPermanent optional parameter.

Advantages of Named Arguments in C#:

There are several advantages of using Named Arguments in C#. Named Arguments in C# are useful when we have methods with multiple optional parameters. They allow us to specify only those arguments that we need and ignore the rest. Furthermore, with named arguments, we improve the code readability and we can pass the arguments out of their positions.

Bullet Points in C# Named Parameters:

1. Using named parameters, we can specify the value of the parameters according to their names, not their order in the method. So, it provides us a facility to not remember parameters according to their order.
2. Named Parameters make our program easier to understand when we are working with a larger number of parameters in our method.
3. Named Parameters in C# are always going to be appeared after the fixed arguments, if we try to provide a fixed argument after the named parameters, then we will get a compilation error.

In the next article, I am going to discuss [C# 7 New Features](#) with Examples. Here, in this article, I try to explain the **Named Parameter in C#** with Examples. I hope you enjoy this Named Parameter in C# with Examples article. I would like to have your feedback. Please post your feedback, question, or comments about this article.